ARUSHA TECHNICAL COLLEGE JUNCTION OF MOSHI-ARUSHA AND NAIROBI ROADS P.O. BOX 296, ARUSHA-TANZANIA
TELEPHONE: +255-27-2503040/2502076, FAX: +255-27-2548337 WEBSITE: http://www.atc.ac.tz, E-MAIL: rector@atc.ac.tz

INFORMATION COMMUNICATION AND DEVELOPMENT DEPARTMENT

ORDINARY DIPLOMA IN COMPUTER SCIENCE

FINAL YEAR PROJECT– 2025/2026

PROJECT TITLE: DESIGN AND DEVELOPMENT OF A DIGITAL PLATFORM FOR CONNECTING UNIVERSITY STUDENTS TO LOCAL PART-TIME AND SKILL-BUILDING JOBS IN TANZANIA

PRESENTED BY: ASHRAFU HUSSEIN HASHIMU
ADMISSION No: 23050502001

NTA LEVEL: 06

Table of Contents

1. PROJECT OVERVIEW

Project Title	Design and Development of a Digital Platform for Connecting Youth to Short-Term Informal Job Opportunities in Tanzania
App Name	Fursafy (Fursa + fy — Swahili for 'Opportunity')
Student	Ashrafu Hussein Hashimu
Admission No.	23050502001
Supervisor	Eng. Abdul Kirobo
Institution	Arusha Technical College (ATC)
Programme	Ordinary Diploma in Computer Science
Academic Year	2025/2026
Core Technologies	Flutter (Frontend), Firebase (Backend — Auth, Firestore, FCM, Cloud Functions)
Methodology	Waterfall Model with SRS (Software Requirements Specification) Approach
Document Scope	Week 1 & Week 2 Design Deliverables (up to 28 April 2026)

1.1 Background

Youth unemployment remains a persistent socio-economic challenge in Tanzania and across East Africa. Each year, approximately 850,000 young people enter the Tanzanian labour market, while only 40,000–50,000 formal employment opportunities are created (ILO, 2022). Despite the availability of short-term and one-time job opportunities — such as technical repairs, cleaning services, tutoring, and construction assistance — access to them is largely unstructured, relying on personal networks, word of mouth, or informal social media groups.

Fursafy addresses this gap by providing a structured, skill-based digital matching platform that connects skilled youth to short-term informal job opportunities posted by individuals and small businesses. Built with Flutter for cross-platform mobile deployment and Firebase for backend services, the platform leverages GeoPoint-based location matching, skill-set filtering, and Firebase Cloud Messaging (FCM) push notifications to deliver real-time job opportunities directly to registered youth.

1.2 Problem Statement

There is no dedicated digital platform in Tanzania that effectively connects skilled youth to short-term informal job opportunities. Current job platforms focus on formal employment,

while informal hiring remains disorganised, inefficient, and unreliable. This contributes to youth unemployment and income instability, despite the availability of work. Fursafy proposes a trust-based, skill-oriented digital matching solution to address this structural gap.

1.3 Research Objectives

- To assess the challenges faced by youth in accessing short-term informal job opportunities.
- To analyse the needs of job providers in sourcing skilled workers for one-time or short-term tasks.
- To design a digital platform that enables skill-based job matching.
- To implement a prototype system that allows job posting and worker discovery.
- To evaluate the usability and effectiveness of the developed platform.

WEEK 1 DELIVERABLES — Due: Friday, 17 April 2026

2.1 Functional & Non-Functional Requirements

Functional Requirements

ID	Feature	Description
FR-01	**User Registration & Login**	Youth and Job Providers can register using email/phone. Firebase Authentication handles secure login.
FR-02	**Profile Management**	Youth can set up profiles with skills, bio, location, and profile photo. Providers set up business profiles.
FR-03	**Job Posting**	Job providers can post short-term job listings specifying title, description, required skills, location (GeoPoint), pay, and deadline.
FR-04	**Job Browsing & Search**	Youth can browse, search, and filter job listings by skill category, location, and pay range.
FR-05	**Skill-Based Job Matching**	A Firebase Cloud Function triggers on each new job post and queries youth whose skills[] overlap jobsRequired[] AND whose GeoPoint is within configurable radius (default: 10km).
FR-06	**Job Application**	Youth can apply for jobs by submitting an application with an optional cover message. Status updates (Pending, Accepted, Rejected) are tracked.
FR-07	**Push Notifications**	Matched youth and job providers receive real-time FCM push notifications for new matches, application status changes, and messages.
FR-08	**Rating & Review System**	After job completion, both parties (youth and provider) can leave ratings (1–5 stars) and written reviews.
FR-09	**Admin Dashboard**	Admin can manage users, monitor job listings, suspend accounts, and view analytics reports.
FR-10	**In-App Notifications Panel**	All notifications are stored in Firestore and accessible from the in-app Notifications screen.

Non-Functional Requirements

ID	Requirement	Specification
NFR-01	**Performance**	The app must load job listings within 2 seconds under normal network conditions. Matching engine must execute within 3 seconds.
NFR-02	**Usability**	The UI must meet ISO 9241-11 (1998) usability standards — effective, efficient, and satisfying for low-literacy users.

ID	Requirement	Specification
NFR-03	**Security**	All user data must be encrypted in transit (HTTPS/TLS). Firebase Security Rules must enforce role-based access.
NFR-04	**Scalability**	The system must support at least 10,000 concurrent users without degradation. Firebase auto-scaling handles backend load.
NFR-05	**Reliability**	System uptime must be at least 99.5%. Firebase SLA provides 99.95% availability.
NFR-06	**Portability**	Flutter enables deployment on both Android and iOS from a single codebase.
NFR-07	**Maintainability**	Code must follow clean architecture principles. Firebase modular SDK enables independent service updates.

2.2 Use Case Diagram

The Use Case Diagram below identifies three primary actors in the Fursafy system: Youth (Job Seeker), Job Provider, and Admin. Each actor interacts with the system through distinct use cases that reflect the core functional requirements defined in Section 2.1.

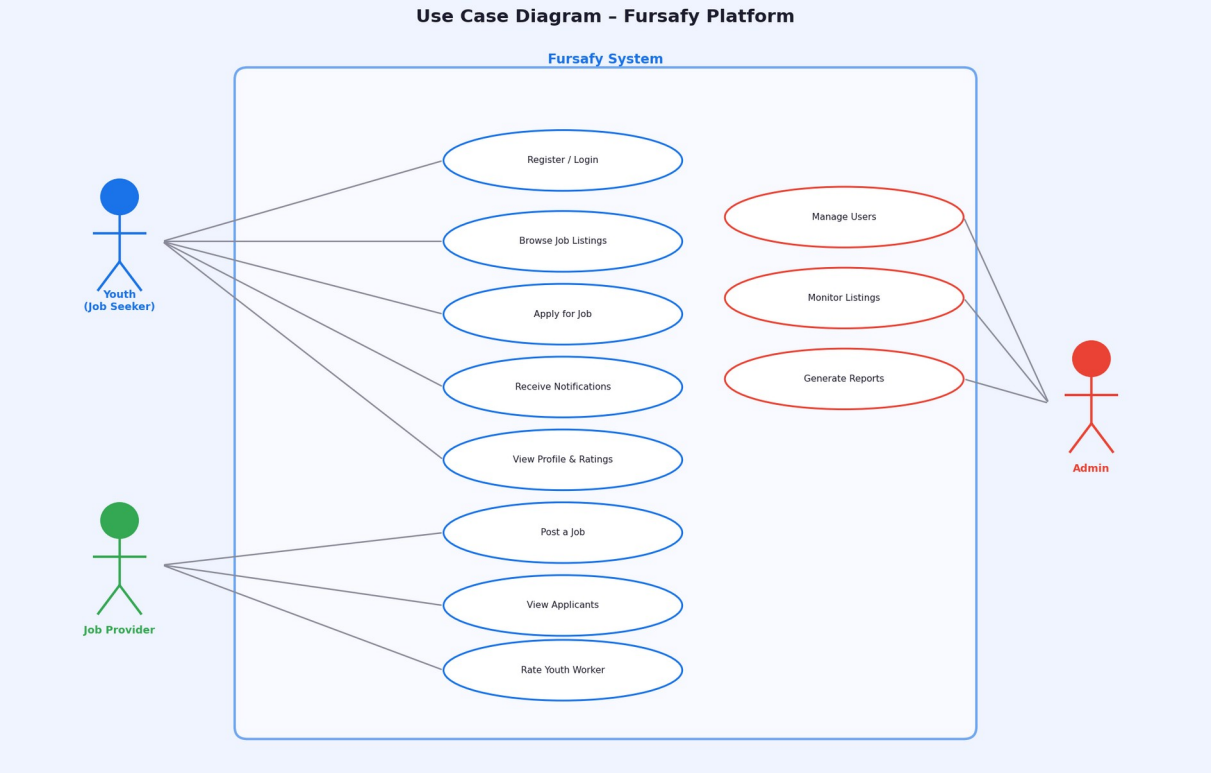

Figure 2.1 – Use Case Diagram: Fursafy Platform (Youth, Job Provider, Admin actors)

The Youth actor can register/login, browse and apply for jobs, receive smart notifications, and view their profile with accumulated ratings. The Job Provider can post jobs, view applicants, and rate youth workers after job completion. The Admin has oversight of user management, listing moderation, and system-wide reporting.

2.3 Entity Relationship Diagram (ERD)

The ERD defines the data model underlying the Fursafy Firestore database. Six primary entities are identified with their attributes and relationships. The skills[] and jobsRequired[] fields are implemented as Firestore array fields to support array-contains-any queries used by the Matching Engine (FR-05).

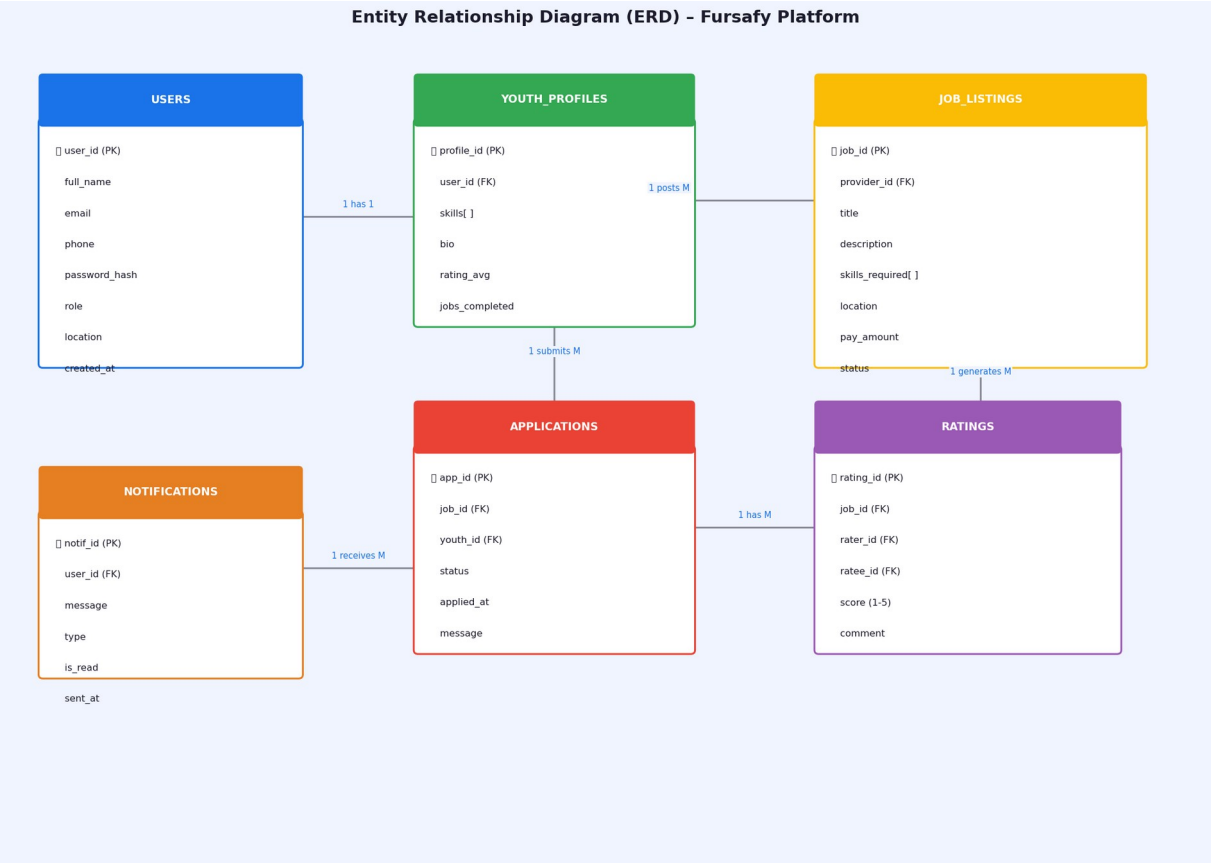

Figure 2.2 – Entity Relationship Diagram: Fursafy Firestore Data Model

Key relationships: A User (role=youth) has exactly one YouthProfile; a User (role=provider) can post many JobListings; a YouthProfile can submit many Applications; a completed job generates Ratings from both parties; all events trigger Notification records.

2.4 DFD Level 0 – Context Diagram

The Context Diagram presents Fursafy as a single process interacting with five external entities: Youth (Job Seeker), Job Provider, Admin, Firebase Backend, and Firebase Cloud Messaging (FCM). It defines the system boundary and the data flows crossing it.

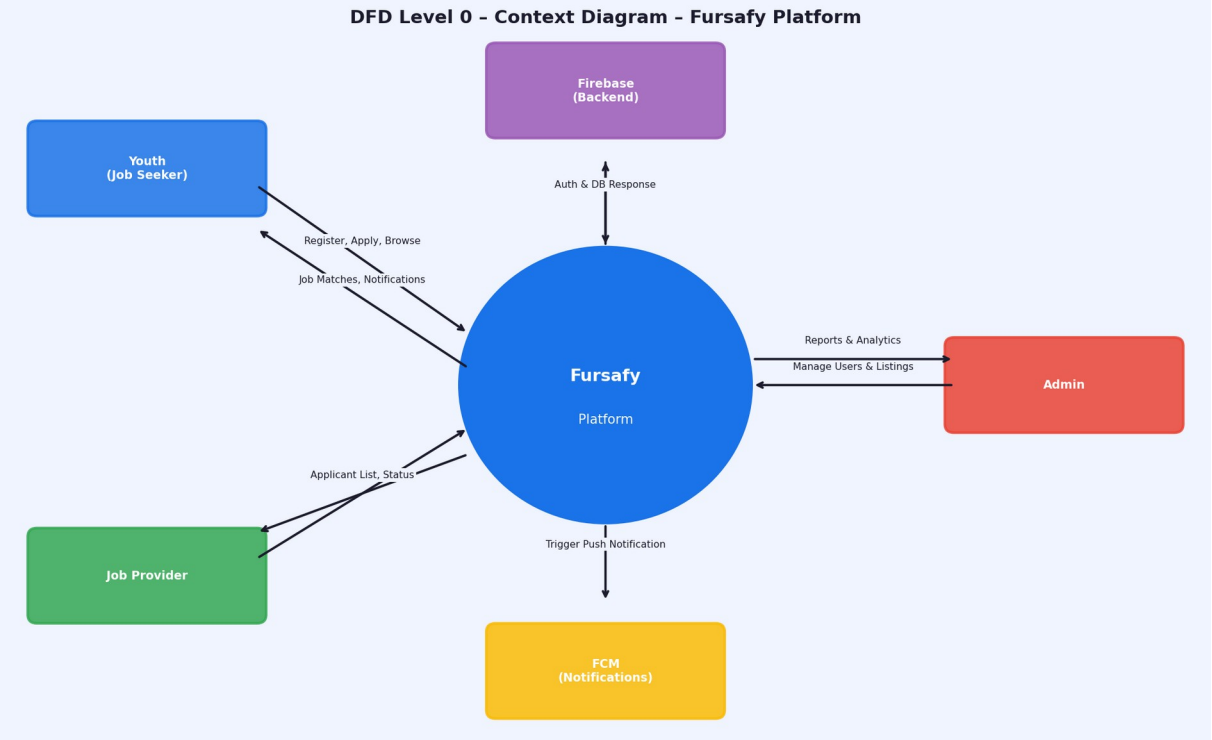

Figure 2.3 – DFD Level 0 (Context Diagram): External entities and data flows

Youth submit registration, job search, and application data to the platform and receive job matches and notifications in return. Job Providers submit job postings and receive applicant lists. The Admin manages the system and receives reports and analytics. Firebase services handle data persistence and push notification delivery.

2.5 Low-Fidelity Wireframes

Low-fidelity wireframes define the structural layout and navigation flow of the Fursafy mobile application without visual styling. Four key screens are presented: Splash/Onboarding, Register/Login, Home Job Feed, and Job Detail with Apply action.

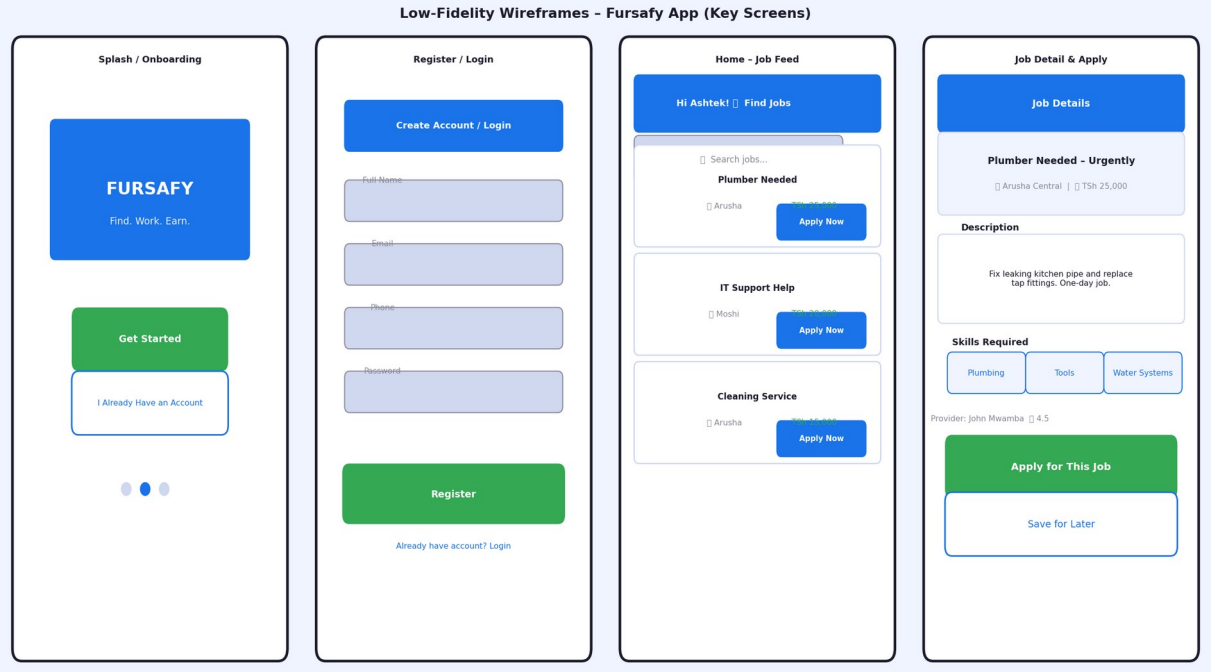

Figure 2.4 – Low-Fidelity Wireframes: Core screens of the Fursafy app (4 screens)

The Splash screen introduces the Fursafy brand with a clear CTA. The Register/Login screen collects essential user data. The Home Feed presents job cards with location, category, and pay at a glance. The Job Detail screen exposes full description, required skills, provider rating, and an Apply button — the central conversion action of the platform.

WEEK 2 DELIVERABLES — Due: Friday, 24 April 2026

3.1 DFD Level 1

The Level 1 DFD decomposes the Fursafy system into seven sub-processes, each with defined data stores and flows. This diagram was developed following the context diagram to provide greater detail on internal system logic.

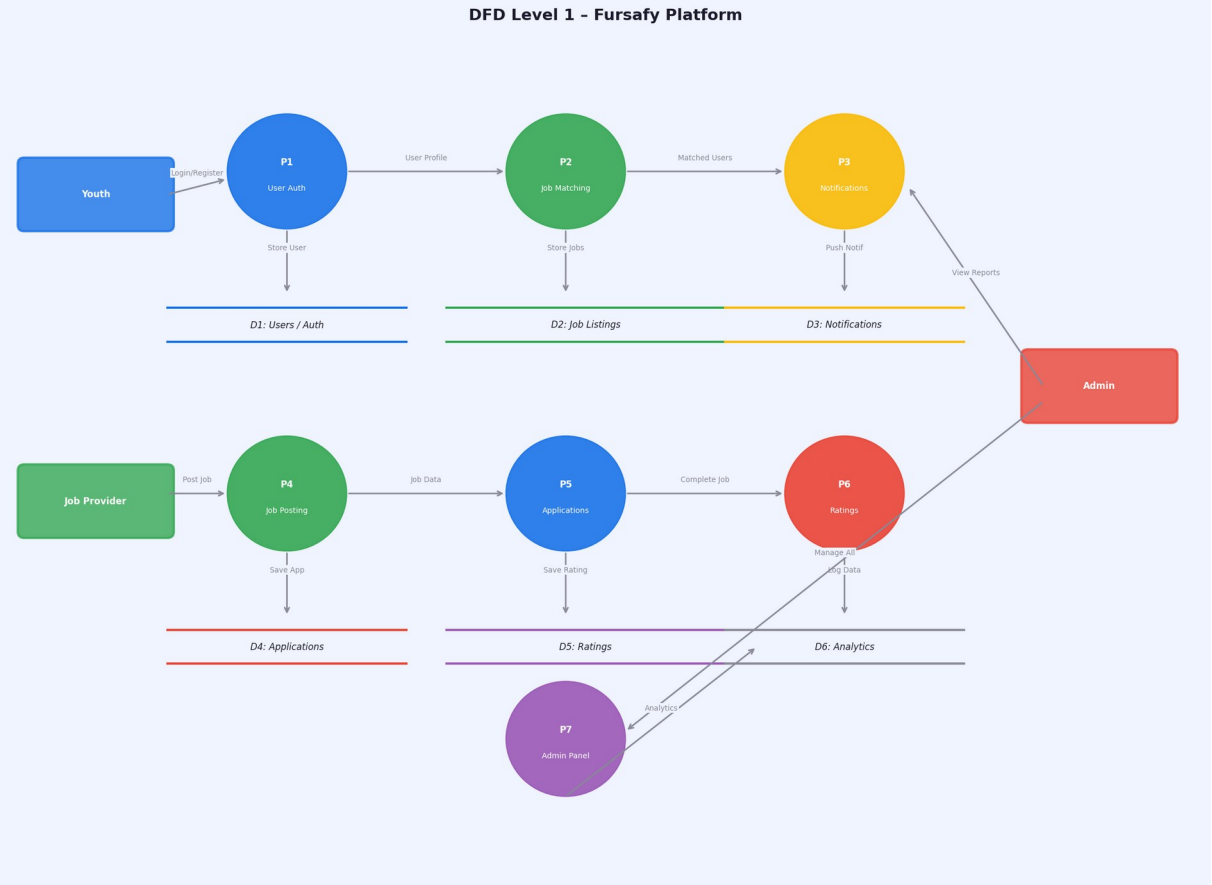

Figure 3.1 – DFD Level 1: Internal process decomposition of the Fursafy Platform

P1 (User Auth) handles registration and token verification against D1 (Users/Auth store). P2 (Job Matching) queries D2 (Job Listings) using skill and location overlap. P3 (Notifications) writes to D3 and triggers FCM. P4–P6 handle Job Posting, Applications, and Ratings. P7 (Admin Panel) has cross-cutting access to all data stores for reporting and governance.

3.2 Class Diagram

The Class Diagram models the object-oriented structure of the Fursafy Flutter application. Seven classes are defined with attributes, methods, and inter-class relationships (associations, dependencies). The MatchingEngine class encapsulates the core algorithm (Sommerville, 2016).

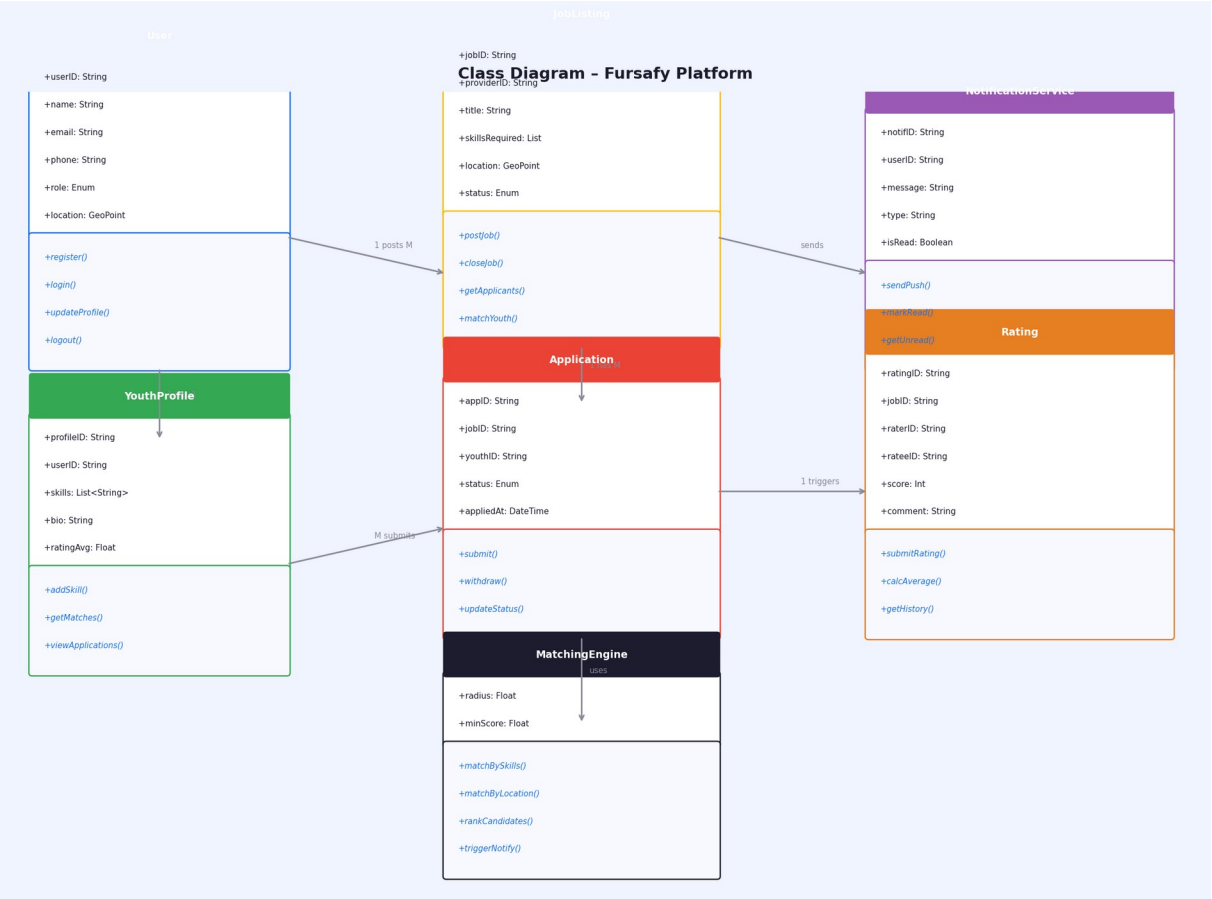

Figure 3.2 – Class Diagram: Object-oriented model of the Fursafy Flutter application

The User class is the base entity, extended by YouthProfile. JobListing is owned by a provider (User) and linked to Application records. The MatchingEngine depends on JobListing and YouthProfile to compute matches, then delegates to NotificationService to deliver FCM push notifications. The Rating class captures bilateral review data after job completion.

3.3 Sequence Diagram

The Sequence Diagram traces the complete interaction flow for the primary use case: a youth user browsing job listings and submitting an application. This covers authentication verification, Firestore query execution, application record creation, and notification dispatch — across four system components (Flutter UI, Firebase Auth, Firestore DB, FCM).

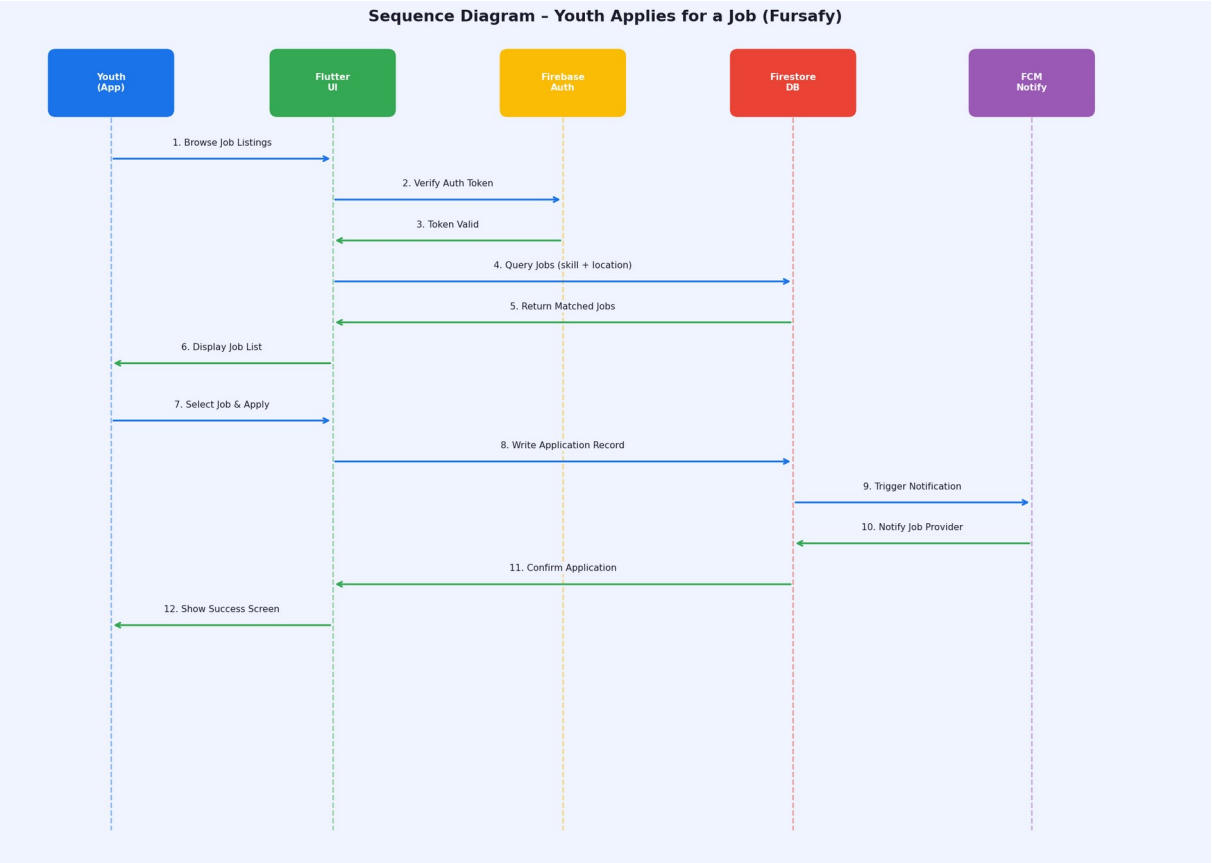

Figure 3.3 – Sequence Diagram: Youth applies for a job (12-step interaction flow)

The flow initiates when the youth browses listings (step 1). The Flutter UI verifies the auth token with Firebase Auth (step 2–3), then queries Firestore for matched jobs (step 4–5). On job selection and application submission (step 7–8), Firestore triggers an FCM notification to the job provider (step 9–10), and the confirmation propagates back to the youth's UI (step 11–12).

3.4 High-Fidelity Wireframes

High-fidelity mockups apply the Fursafy visual identity — dark navy background (#0A1628), primary blue (#1A73E8), and accent green (#4CAF50) — to the structural wireframes from Week 1. Four screens are presented: Onboarding, Job Feed, Job Detail, and Profile with Ratings.

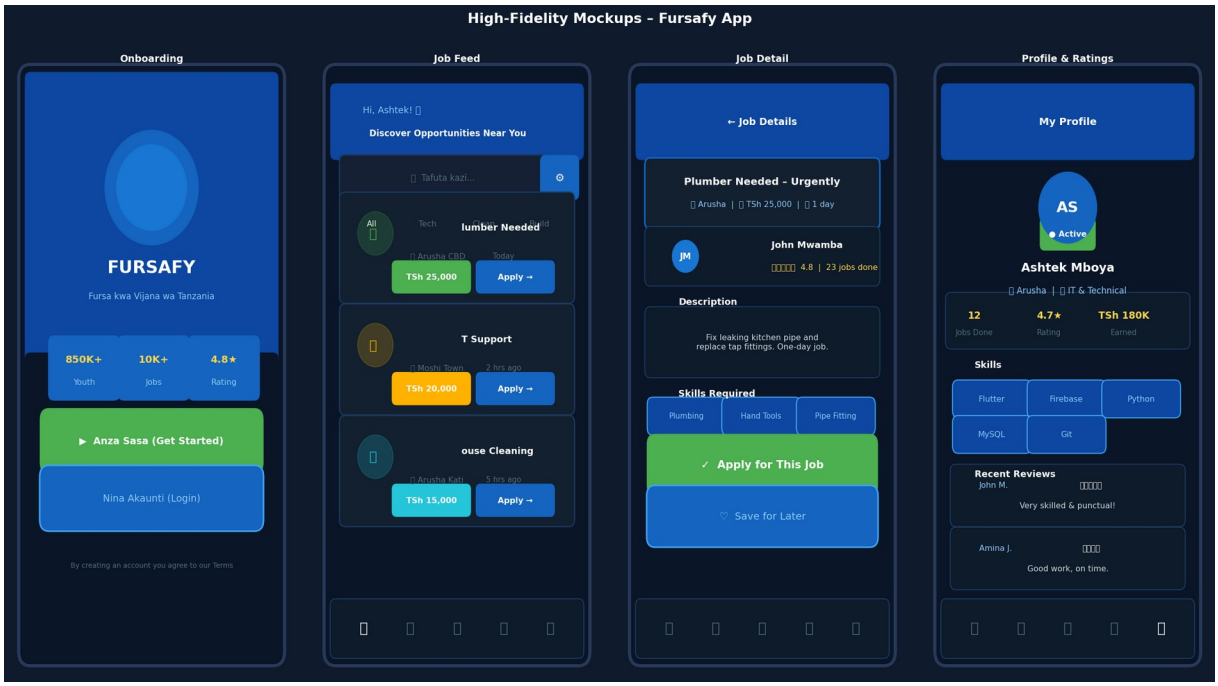

Figure 3.4 – High-Fidelity Mockups: Fursafy app (Onboarding, Feed, Detail, Profile)

The Onboarding screen presents platform statistics and bilingual CTAs (Swahili/English). The Job Feed employs card-based design with colour-coded category tags and dual-action buttons (Apply / Save). The Job Detail screen exposes provider ratings, skill chips, and the primary Apply CTA. The Profile screen displays earned statistics, skills, and recent reviews — building trust between youth and job providers.

3.5 Technology Stack Justification

The selection of Flutter and Firebase as the core technology stack was guided by the principles of Design Science Research (Hevner et al., 2004) and the project's constraints of cost, time-to-market, and Tanzanian mobile infrastructure realities.

Technology	Role in Fursafy	Justification
Flutter (Dart)	Cross-platform mobile frontend (Android & iOS)	Single codebase reduces development time by ~40%. Hot reload accelerates prototyping. Supports offline-first UI patterns critical for low-connectivity areas in Tanzania.
Firebase Firestore	NoSQL real-time database for jobs, users, applications, ratings	Real-time sync via Firestore listeners eliminates polling. Array-contains-any queries directly support skills-based job matching. Free Spark tier suits academic prototype.
Firebase Auth	Secure user authentication (email/password, phone)	Handles token management, session persistence, and password reset out-of-the-box. Reduces custom auth development to zero.
Firebase Cloud Functions	Serverless backend — Matching Engine, notification triggers	Matching algorithm runs server-side (not on device), ensuring consistency and security. Triggered automatically on new job post (Firestore onCreate trigger).
Firebase FCM	Push notification delivery to matched youth and providers	Free, reliable push delivery at scale. Integrates natively with Flutter via firebase_messaging package. Critical for real-time job matching UX.
Google Maps SDK	Location display and GeoPoint capture for job and user location	GeoPoint stored in Firestore enables proximity-based matching. Maps SDK provides familiar location picker UI for non-technical users.

Matching Algorithm Detail

The Matching Engine is implemented as a Firebase Cloud Function that triggers on every new JobListing document created in Firestore (functions.firestore.document('jobs/{jobId}').onCreate). The algorithm executes the following steps:

- Step 1: Read the new job document — extract jobsRequired[] (skills array) and location (GeoPoint).
- Step 2: Query the youth_profiles collection for all documents where skills array-contains-any of the jobsRequired values (Firestore compound query).
- Step 3: For each candidate youth, compute the Haversine distance between their GeoPoint and the job GeoPoint. Retain only youth within configurable radius (default: 10km).
- Step 4: Rank candidates by: (a) number of matching skills (descending), then (b) distance (ascending), then (c) rating average (descending).

- Step 5: For each ranked match, write a notification document to Firestore (notifications/{userId}), which triggers an FCM push notification to the youth's device token.

3.6 System Architecture Diagram

The architecture follows a four-layer model: Presentation (Flutter), Application (Firebase Cloud Functions / Business Logic), Data (Firebase Services), and External/Infrastructure. This layered approach supports separation of concerns, testability, and independent scalability of each tier (Pressman, 2014).

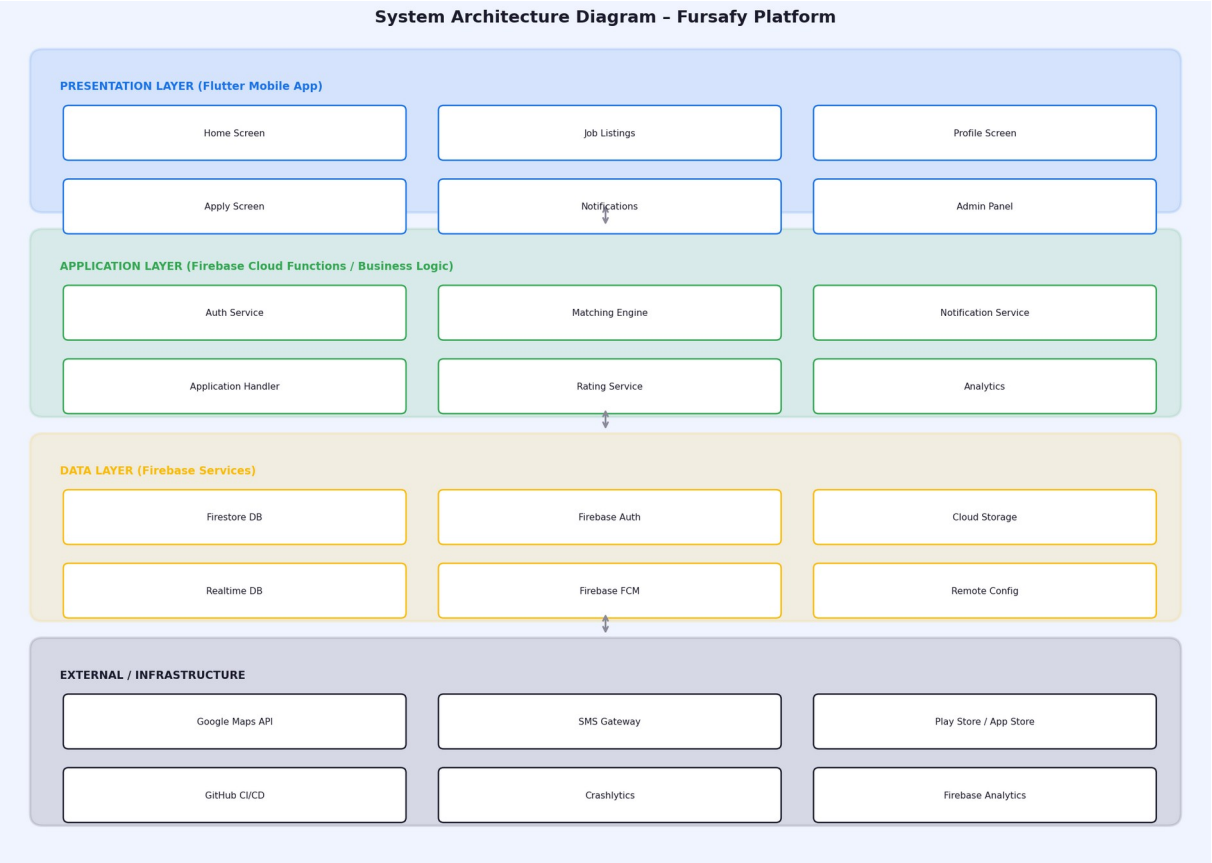

Figure 3.5 – System Architecture Diagram: Four-layer Fursafy architecture

The Presentation Layer comprises all Flutter screens (Home, Job Listings, Profile, Notifications, Admin Panel). The Application Layer encapsulates the Auth Service, Matching Engine, Notification Service, Application Handler, Rating Service, and Analytics. The Data Layer leverages Firestore (primary DB), Firebase Auth, Cloud Storage (profile images), Realtime DB (presence/chat), FCM, and Remote Config. The Infrastructure Layer includes Google Maps API, SMS Gateway fallback, Play Store/App Store deployment, GitHub CI/CD, and Crashlytics for production monitoring.

4. DELIVERABLES SUMMARY

The table below summarises all 11 design deliverables required across Week 1 (due 17 April 2026) and Week 2 (due 24 April 2026), compiled as of 28 April 2026.

Week 1 — System Design Foundations (Due: 17 April 2026)

#	Deliverable	Description	Status
1	**Functional & Non-Functional Requirements**	10 Functional (FR-01 to FR-10) and 7 Non-Functional (NFR-01 to NFR-07) requirements documented with IDs.	**Done**
2	**Use Case Diagram**	3 actors (Youth, Provider, Admin), 9 use cases, relationships and system boundary defined.	**Done**
3	**Entity Relationship Diagram (ERD)**	6 entities: Users, YouthProfile, JobListings, Applications, Ratings, Notifications — with attributes and relationships.	**Done**
4	**DFD Level 0 (Context Diagram)**	5 external entities, data flows crossing system boundary, complete context view.	**Done**
5	**Low-Fidelity Wireframes**	4 key screens: Splash, Register/Login, Job Feed, Job Detail — structural layout without styling.	**Done**

Week 2 — Design Finalization & Approval (Due: 24 April 2026)

#	Deliverable	Description	Status
6	**DFD Level 1**	7 sub-processes (P1–P7), 6 data stores (D1–D6), internal data flows, actor connections.	**Done**
7	**Class Diagram**	7 classes with attributes, methods, and inter-class relationships. MatchingEngine class included.	**Done**
8	**Sequence Diagram**	12-step interaction flow for Youth applies for a Job — across Flutter UI, Firebase Auth, Firestore, FCM.	**Done**
9	**High-Fidelity Wireframes**	4 fully styled mockups: Onboarding, Job Feed, Job Detail, Profile & Ratings.	**Done**
10	**Tech Stack Justification**	Flutter + Firebase stack justified across 6 technologies with matching algorithm detail.	**Done**
11	**System Architecture Diagram**	4-layer architecture: Presentation, Application, Data, Infrastructure — with component breakdown.	**Done**

References

Google. (2023). Firebase documentation. Google LLC. https://firebase.google.com/docs

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75–105.

ILO. (2022). World employment and social outlook: Trends 2022. International Labour Organization.

ISO 9241-11. (1998). Ergonomic requirements for office work with visual display terminals (VDTs) — Part 11: Guidance on usability. International Organization for Standardization.

Pressman, R. S. (2014). Software engineering: A practitioner's approach (8th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Oates, B. J. (2006). Researching information systems and computing. Sage Publications.

Creswell, J. W. (2014). Research design: Qualitative, quantitative, and mixed methods approaches (4th ed.). Sage.

Saunders, M., Lewis, P., & Thornhill, A. (2019). Research methods for business students (8th ed.). Pearson.